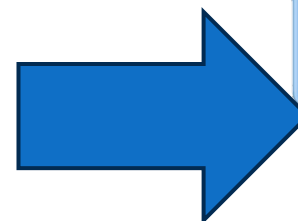


React Nativeを使用した クロスプラットフォーム開発

クロスプラットフォーム開発とは

- ▶ 1つのソースコードを基に複数のプラットフォーム(Android,iOSなど)に対応したアプリケーションを開発する手法
- ▶ 利点
 - ▶ プラットフォームごとの独自言語を習得する必要がない
 - ▶ 複数プラットフォームの同時開発が可能
- ▶ 欠点
 - ▶ プラットフォーム固有の機能には対応できない(iPhone LiDARなど)

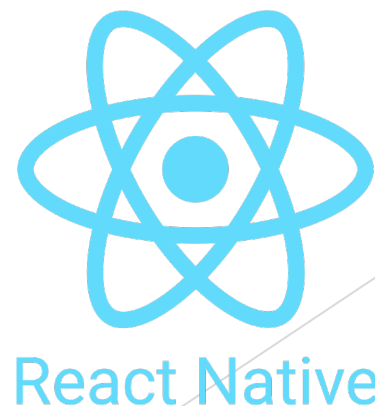


代用的なフレームワーク

Flutter	React Native	.NET MAUI	Unity
Google	Meta(旧Facebook)	Microsoft	Unity Technologies
オープンソース	オープンソース	オープンソース	プロプライエタリ
Dart	JavaScript TypeScript	C# F#	C#
独自UI	ネイティブUI	ネイティブUI 独自UI	独自UI
Android iOS Web Windows Mac Linux	Android iOS Web Windows Mac	Android iOS Windows Mac	Android iOS Web Windows Mac Linux ゲーム機

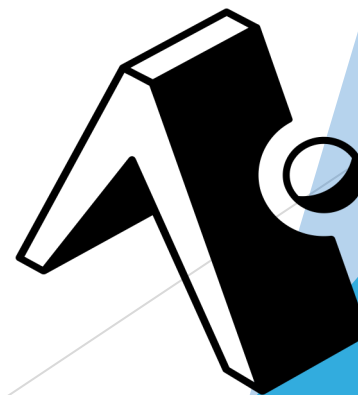
React Native

- ▶ Meta(旧Facebook)社が開発したクロスプラットフォーム開発フレームワーク
- ▶ Reactライブラリの基本構造を踏襲している
 - ▶ 開発言語がJavaScript/TypeScript
 - ▶ コンポーネント指向
 - ▶ `React.Component<P,S>/React.FC<P>/JSX.Element`でコンポーネントを開発する
 - ▶ Reactからの学習コストが低い
- ▶ ホットリロードによる開発が可能(コンパイル不要)



Expo

- ▶ React Nativeアプリを開発するための便利機能を集めた開発プラットフォーム
- ▶ React Nativeを素で構築するよりも開発が容易になる
- ▶ いくつかのネイティブライブラリが使用できない制約がある
- ▶ Expo自体にアプリケーション配信機能がある(Expo Application Services)
 - ▶ EAS Build: パッケージビルド→アプリケーションへの署名
 - ▶ EAS Submit: Apple StoreやGoogle Playにアプリケーションを提出



Expo 実行モード

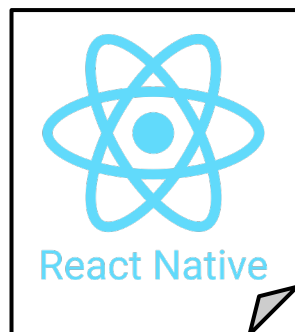
Expo Goモード

- アプリのコードをスマホ側に転送し、スマホにインストールしたExpo Goアプリで実行するモード
- コードの修正を即時にアプリに適用可能(ホットリロード)
- デバッグモードが実行可能
- ネイティブライブラリが利用できない

Development Buildモード

- アプリのコードから実行可能なバイナリをコンパイルし、バイナリをスマホに転送して実行するモード
- ネイティブライブラリを含め、全ての機能が利用可能
- Expo Goアプリを必要としない
- デバッグモードが使用できない
- コード修正後、コンパイルが必要になる

Expo Goモード



ソースを転送



スマートフォン
(Android,iOS)



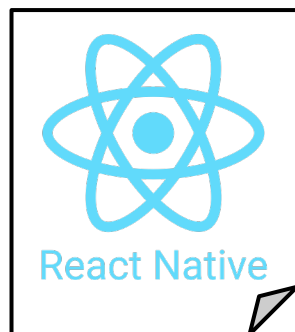
アプリ内部で実行



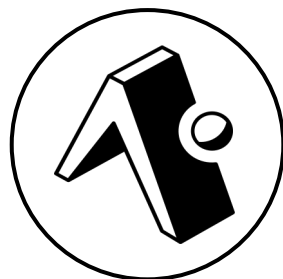
アプリケーション



Development Buildモード



コンパイル



バイナリを実行

Expoが各プラットフォームの
バイナリを生成



スマートフォン
(Android,iOS)

開発環境構築

- ▶ 必須環境
 - ▶ Node.js
 - ▶ watchman (ホットリロードで使用)
- ▶ プラットフォームごとの環境
 - ▶ iOS
 - ▶ Xcode
 - ▶ Xcode Command Line Tool
 - ▶ Ruby + CocoaPods
 - ▶ CocoaPods: Xcodeのライブラリ管理ツール(Ruby Gemでインストール)
 - ▶ Android
 - ▶ **Java 17** (Android 15ではJava 21は未対応)
 - ▶ Android Studio
 - ▶ Android SDK Platform 35
 - ▶ Android SDK Command-line Tools

開発環境構築

▶ あると便利

▶ yarn

- ▶ npmのようなパッケージ管理ツール
- ▶ npmより使いやすい(個人的主観)

▶ エミュレータ

▶ iOS Simulator

- ▶ Xcodeから必要なiOS Simulator用Deviceをインストール
- ▶ iOS Simulatorから使用できるように設定する

▶ Android Emulator

- ▶ Android Studioから必要なAndroid Virtual Deviceをインストール
- ▶ Android Emulatorから使用できるように設定する
- ▶ M1 Mac用のAndroid Emulatorについて、最新バージョン(34系)は正常に動作しないので、バージョンダウン(33系)する必要がある
(https://developer.android.com/studio/emulator_archive)

新規プロジェクトを作成

- ▶ yarn create コマンドを利用してExpoプロジェクトを新規に作成

```
% yarn create expo-app ExpoApp -template blank-typescript
```

```
.....
```

```
.....
```

Expoプロジェクト名

templateオプションで
TypeScriptテンプレートを指定

✅ Your project is ready!

To run your project, navigate to the directory and run one of the following yarn commands.

- cd ExpoApp
- yarn android
- yarn ios
- yarn web

✨ Done in 18.27s.

```
%
```

新規プロジェクトを作成

▶ 生成されたファイル構造

```
% tree .
```

```
├── App.tsx
├── app.json
├── assets
│   ├── adaptive-icon.png
│   ├── favicon.png
│   ├── icon.png
│   └── splash.png
├── babel.config.js
├── node_modules
│   └── .....
├── package.json
├── tsconfig.json
└── yarn.lock
```

Expoで作成するアプリケーションの本体

Expo Router

- ▶ 画面構造や画面遷移を管理するファイルベースのルータ
- ▶ ファイル構造をそのままルーティングに設定できる
 - ▶ “app”フォルダ以下にすべての画面を配置
 - ▶ 配置されたフォルダ構造そのままに画面にアクセス可能
- ▶ React Routerの強化版
 - ▶ `<Routes><Route ... /><Route ... />...</Routes>`を定義する必要がない
- ▶ 特殊なファイル名
 - ▶ `index.tsx`
 - ▶ フォルダを指定してアクセスした場合に読み込まれる画面
 - ▶ `_layout.tsx`
 - ▶ 画面レイアウトとナビゲーション方法を定義する

新規プロジェクトを作成(Expo Router対応)

▶ 生成されたファイル構造

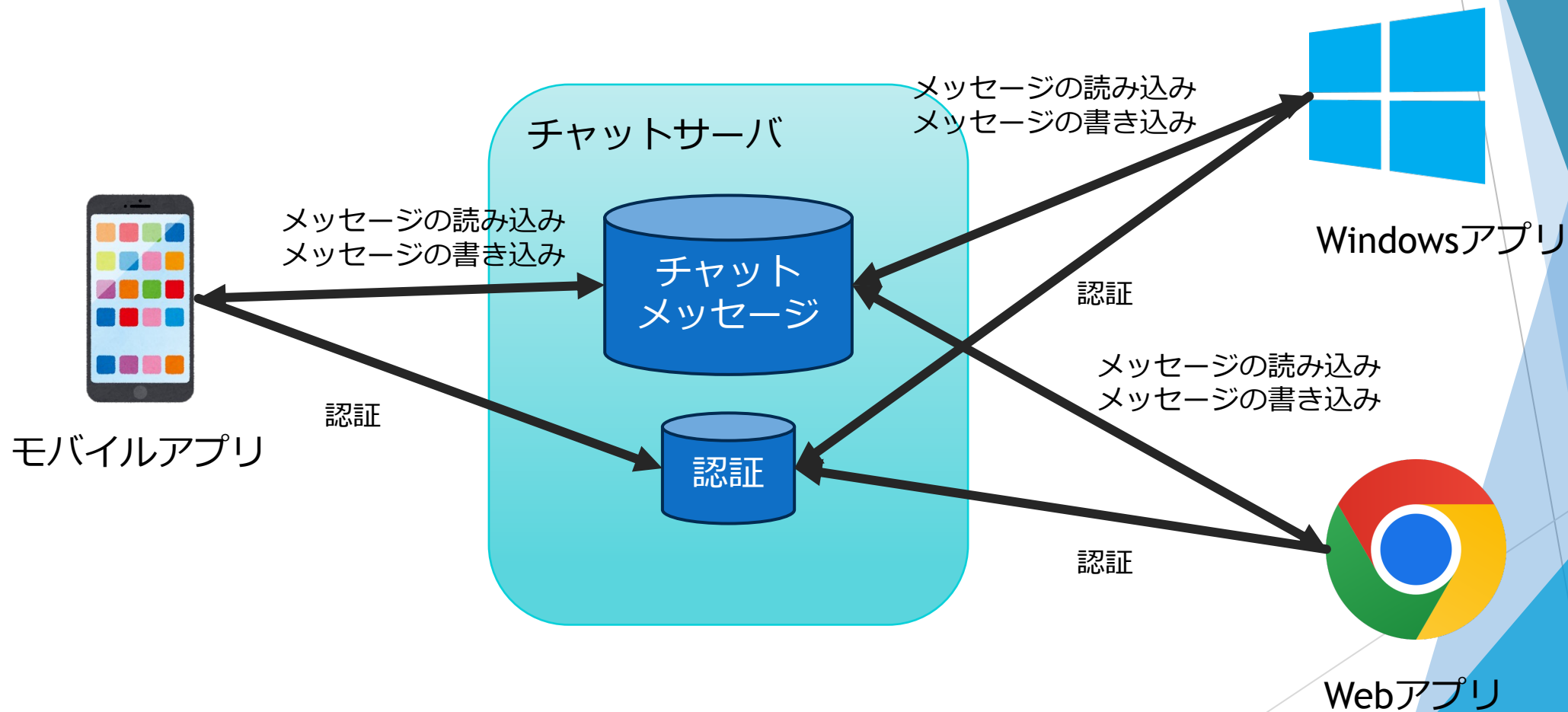
```
% tree .
```

```
├── app
│   ├── _layout.tsx
│   └── index.tsx
├── app.json
├── assets
│   └── .....
├── babel.config.js
├── node_modules
│   └── .....
├── package.json
├── tsconfig.json
└── yarn.lock
```

Expo Routerが使用する
アプリケーション本体

構築するアプリケーションの概要

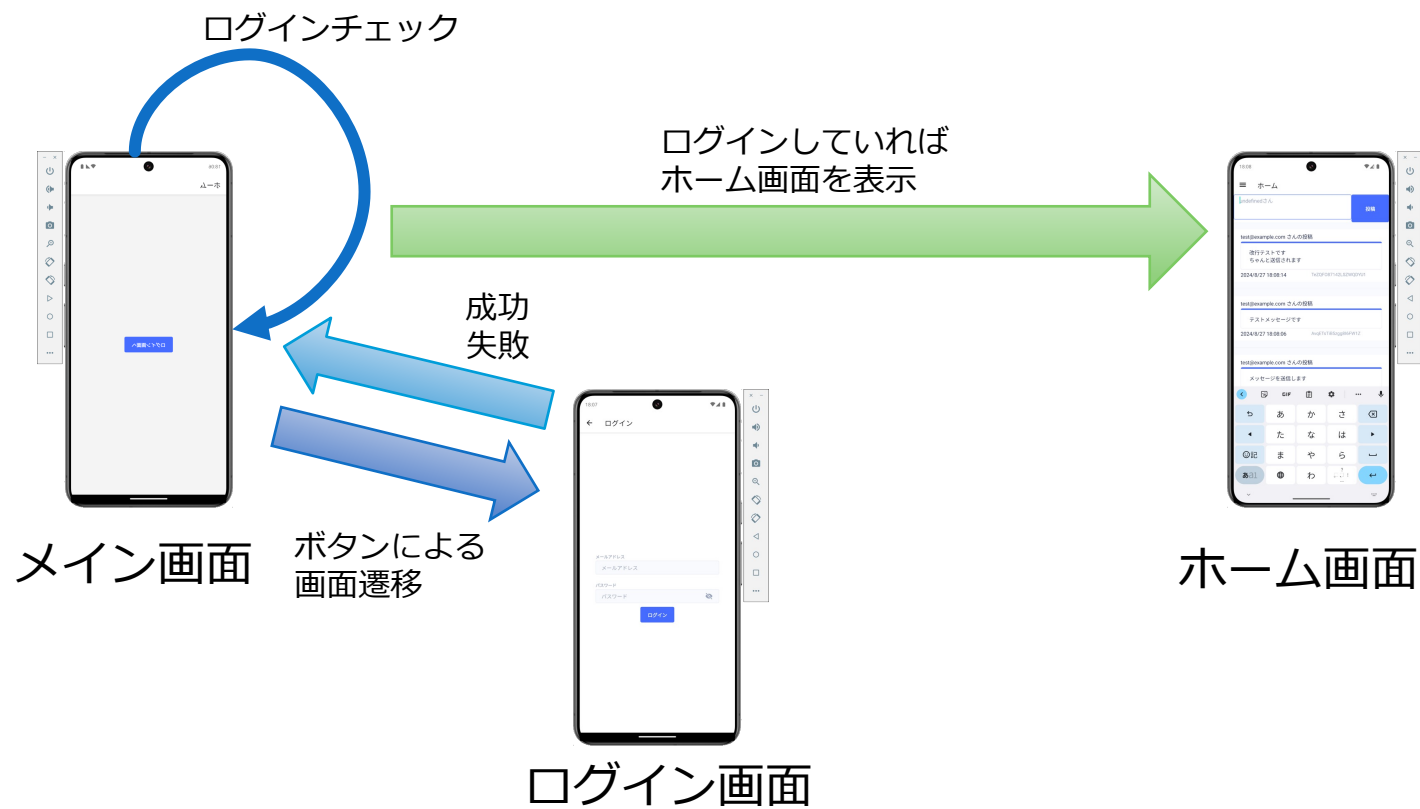
チャットアプリ



サーバにFirebaseを利用する



チャットアプリ 画面遷移



Firestoreのデータ構造

/messages

- sender_id: 送信者ID
- sender_name: 送信者名
- message_type: メッセージの種類
- content: メッセージ
- media_url: メディアURL
- timestamp: 登録日時(サーバ打刻)

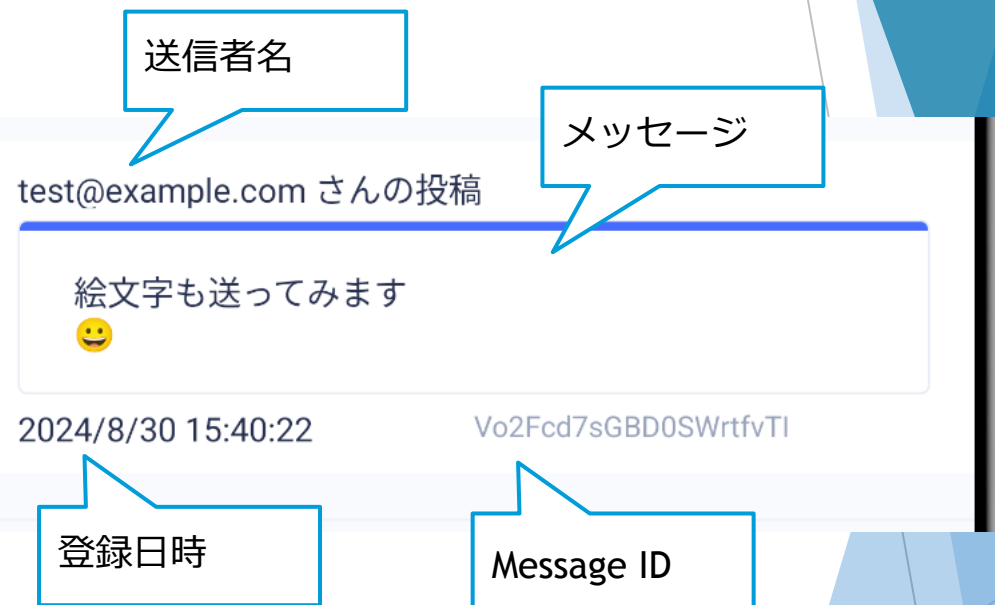
/users

- name: 名前
- profile_picture: 利用者の画像
- bio: 自己紹介文

ホーム画面の構成

メッセージ
入力

メッセージ
一覧表示
(新しい順)



React Native(Expo)にFirebaseを組み込む

Firebase JS SDK

- Webサイト構築時などに利用するJavaScript用のFirebaseライブラリをそのまま利用
- Firebaseの基本的な機能に対応(Firestore, Functions, Auth など)
- Pure JavaScriptで構築されているため、Expo Goモードで使用可能
- ネイティブコードを必要とする機能が使用できない(Analytics など)

React Native Firebase

- Firebaseが提供するネイティブライブラリをReact Nativeから利用できるようにラップしたライブラリ
- Firebaseが提供するすべての機能に対応
- ネイティブコードで実行する必要があるため、Expo Goモードでは利用できない(コンパイルが必要になる)

基本的には Firebase JS SDKを使用する
必要な場合に限り React Native Firebaseを使用する

Firebase Authの永続化

- ▶ Firebase V10.3以降、認証の永続化ストレージを指定することが推奨される
- ▶ デフォルトの永続化ストレージは **IndexedDBLocalPersistence**
 - ▶ このストレージはWebブラウザ用であり、スマホアプリでは有効にならない
- ▶ React Nativeでは永続化ストレージとして **AsyncStorage** が用意されている
- ▶ Firebase Authの永続化ストレージに **AsyncStorage** を使用するように対応する

```
import AsyncStorage from "@react-native-async-storage/async-storage";

const app = initializeApp({....});
const auth = initializeAuth(app, {
  persistence: getReactNativePersistence(AsyncStorage),
});
```

デモ

https://github.com/T-YOSHIZAWA/MIJS_2024_summer/tree/main

プラットフォームごとの違い

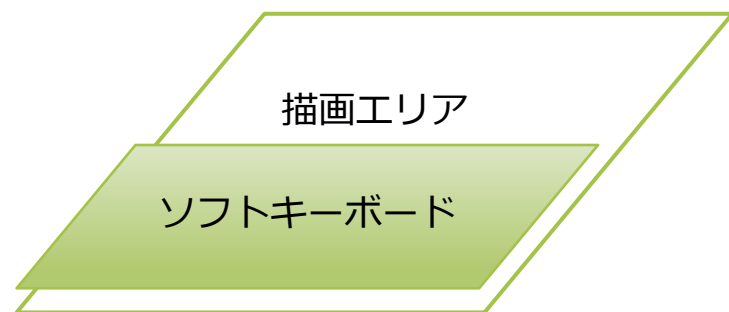
- ▶ ホストOSへのアクセス方法が異なる
 - ▶ デバッグ時にサーバエミュレータをローカルで起動する場合に問題になる
 - ▶ iOS: 127.0.0.1(`localhost` でアクセス可能)
 - ▶ Android: 10.0.2.2 (`localhost` ではアクセスできない)
- ▶ プラットフォームごとの差を制御するための設定
 - ▶ Platform.OS
 - ▶ ios/android/windows/macos/web のどれかが設定されている
 - ▶ if/switch 構文などでプラットフォームごとの制御を行う

プラットフォームごとの違い

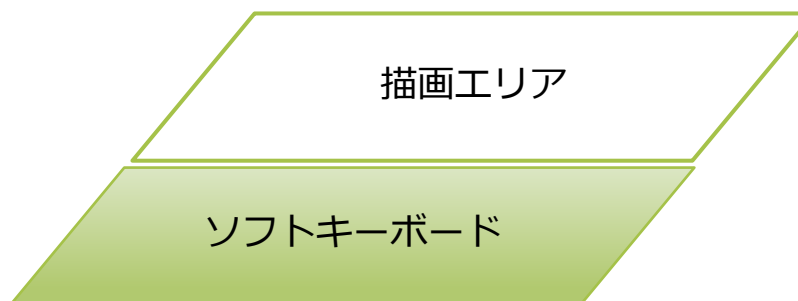
```
//エミュレータの設定
//開発で実行する場合のみ設定
if (process.env.EXPO_PUBLIC_LOCAL === "true") {
  //プラットフォームによってエミュレータのIPアドレスが異なる
  if (Platform.OS === "android") {
    //Androidの場合: EmulatorのIPアドレスは10.0.2.2
    // https://developer.android.com/studio/run/emulator-networking?hl=ja
    connectAuthEmulator(auth, 'http://10.0.2.2:9099');
    connectFirestoreEmulator(db, '10.0.2.2', 8080);
  } else {
    //Web,iOSの場合: EmulatorのIPアドレスは127.0.0.1
    // iOS Simulatorはホストコンピュータのネットワークをそのまま利用するため、
    // localhostが利用可能
    connectAuthEmulator(auth, 'http://127.0.0.1:9099');
    connectFirestoreEmulator(db, '127.0.0.1', 8080);
  }
}
```

プラットフォームごとの違い

- ▶ ソフトキーボードの表示方法が異なる
 - ▶ iOS: 描画エリアに被せる形で表示する
 - ▶ ソフトキーボードを表示したときに、入力項目がソフトキーボードによって隠れてしまう可能性がある
 - ▶ Android: 描画エリアの一部として表示する
 - ▶ アプリの描画エリアが小さくなるため、レイアウトが崩れる可能性がある



iOS



Android

ソフトキーボードを表示したときの動作の違いを意識する

プラットフォームごとの違い

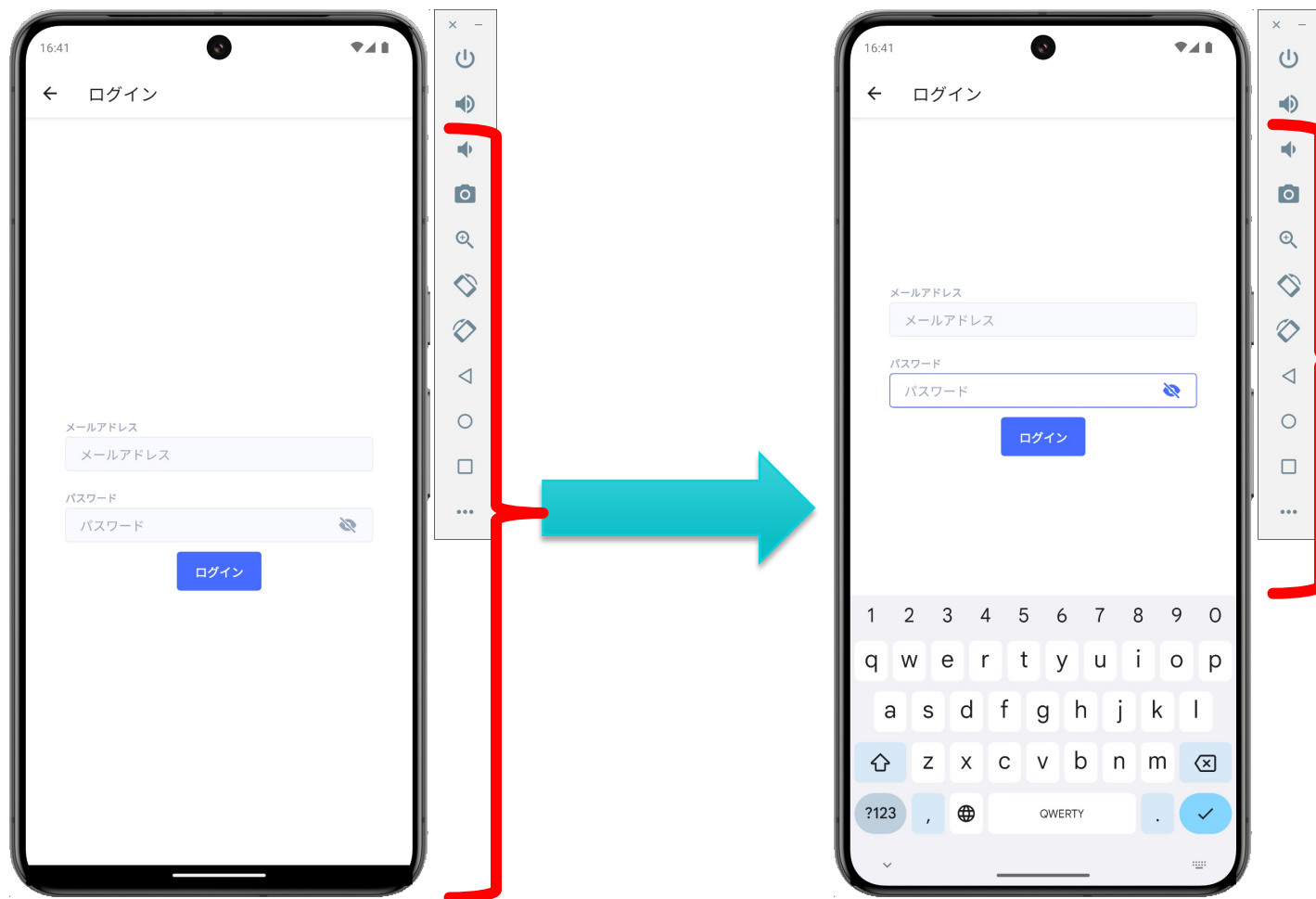
▶ iOSの場合



入力項目にキーボードが被ってしまい、入力欄が見づらくなる

プラットフォームごとの違い

▶ Androidの場合



キーボードが表示された分だけ
描画エリアが小さくなる
レイアウトが崩れてしまう可能性
があるため、注意が必要

まとめ

まとめ

- ▶ React Nativeを利用することで、1つのソースから複数のプラットフォーム向けのアプリを構築することができる
- ▶ ExpoはReact Nativeを拡張する開発プラットフォーム
 - ▶ React Nativeには無い、便利なライブラリや機能が用意されている
 - ▶ Expo Routerが非常に便利 (Reactに欲しい)
- ▶ React NativeとFirebaseの連携は非常に簡単
 - ▶ JavaScript用のFirebaseライブラリがそのまま利用できる
 - ▶ ネイティブライブラリも利用可能(コンパイルが必要になる)
- ▶ プラットフォームごとに動作が異なる部分がある
 - ▶ ホストOSへのアクセス方法
 - ▶ ソフトキーボードの表示方法

Thank you

GitHubにて公開

https://github.com/T-YOSHIZAWA/MIJS_2024_summer/tree/main